System Modeling and Design

Gym Membership Management System

Part 3 of 9 • Use Case, Class, Activity, Sequence, and ER Diagrams

Project	Gym Membership Management System	**Version**	1.0
Prepared by	Moeen Rebar, Shady Jamil, Bilal Taha	**Document**	Models (Part 3)

About This Document

This part turns the requirements in the SRS (Part 2) into models that show how the Gym Membership Management System is structured and how it behaves. Each model has a different purpose and answers a different question. Together they connect the requirements to the design, the code, and the database that the team will build.

The Five Models at a Glance

Model	Purpose: what question does it answer?	Based on (SRS)
1. Use Case Diagram	Who uses the system and what can each user do? It checks that every user requirement has a matching function.	UR-01 to UR-08, FR-01 to FR-26
2. Class Diagram	What are the main classes, what data and operations do they have, and how are they related? It is the starting point for the program code.	FR-05 to FR-26
3. Activity Diagram	What are the steps of an important workflow (renewing a subscription and recording a payment), who does each step, and where are the decisions?	FR-11, FR-12, FR-15, FR-16
4. Sequence Diagram	How do the user interface, the application logic, and the database exchange messages over time in one scenario (recording a payment)?	FR-15, FR-16, NFR-09
5. ER Diagram	Which tables, keys, and relationships will the database have? It guides the database design.	FR-05 to FR-25, NFR-05, NFR-09

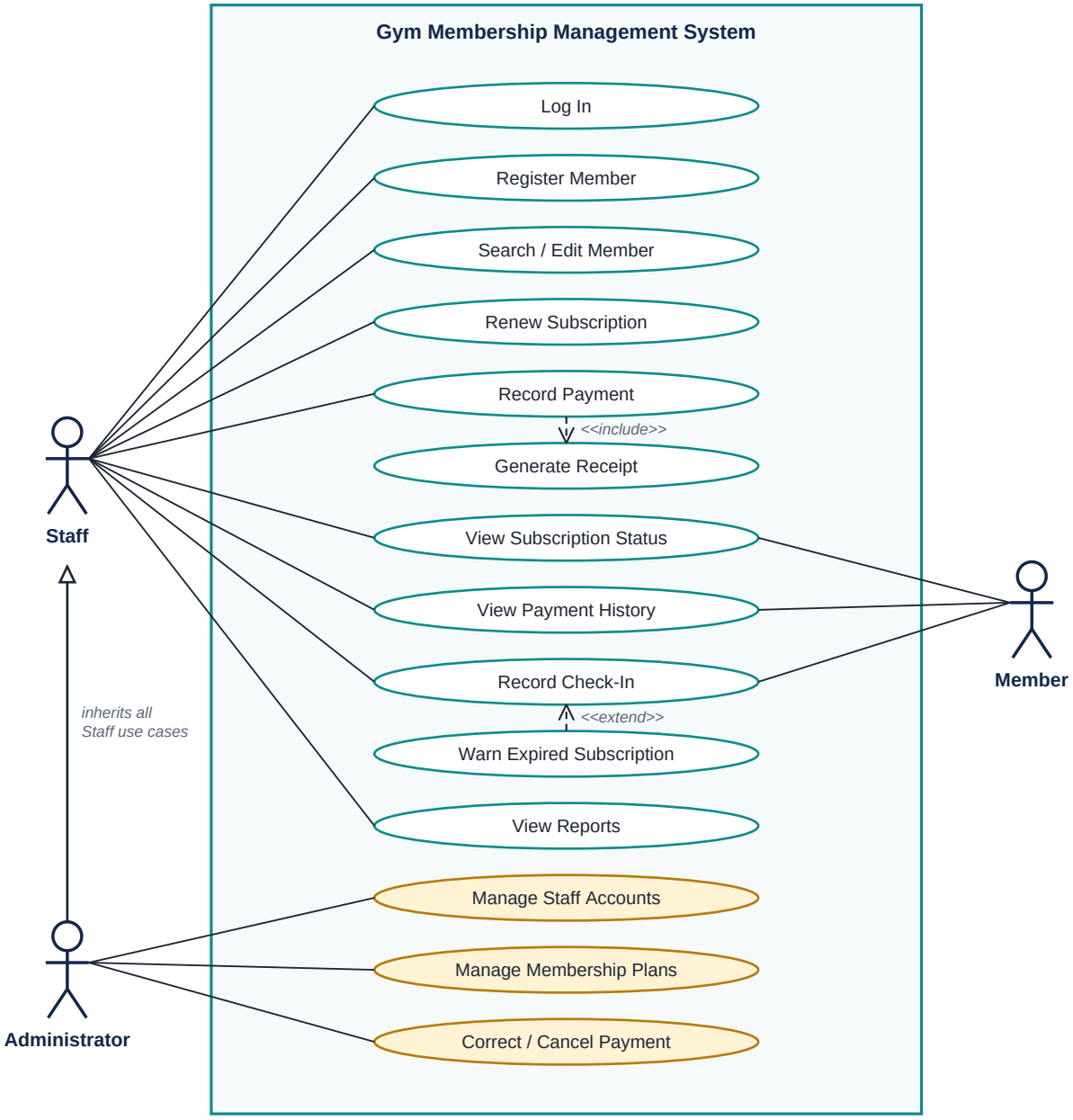
How the Models Support the Engineering of the System

- **Requirements to models:** every diagram traces back to numbered requirements from the SRS, so nothing is drawn without a reason.
- **Models to implementation:** the classes and the ER diagram guide the program code and the database; the activity and sequence diagrams guide how the logic is written.
- **Models to testing:** use cases and the activity flow become test scenarios in the testing part of the project.
- **Models to architecture:** the layers shown in the sequence diagram (interface, logic, database) lead into the architecture design in Part 4.

> **Consistent with the SRS:** Members do not log in (they get information through staff), the Administrator can do everything Staff can do, and payments can only be corrected by the Administrator with a log entry.

1 Use Case Diagram

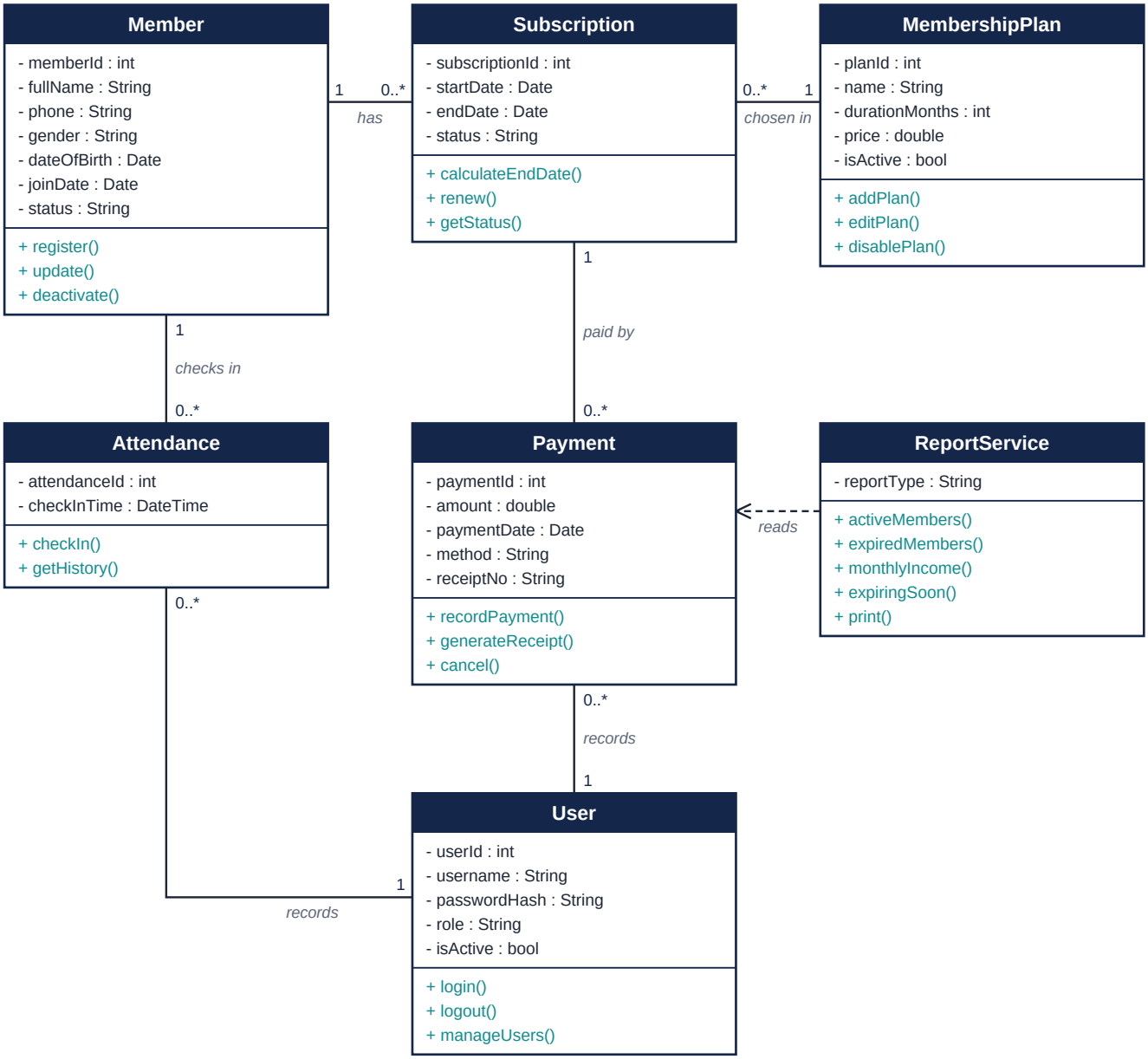
Purpose. The use case diagram shows the system from the users' point of view. It lists the actors (the people who interact with the system) and the functions each one can use. It is the easiest way to confirm that every user requirement in the SRS has a matching function.



- **Actors:** Staff (receptionist), Administrator, and Member. The Administrator inherits every Staff use case (hollow arrow) and adds the orange ones: managing staff accounts and plans, and correcting payments.
- **Member** is an external actor who does not log in; the member checks in and gets status and history through staff.
- **<<include>>:** a receipt is always generated when a payment is recorded. **<<extend>>:** the warning appears only when an expired member tries to check in. All use cases except Log In require a successful login.

2 Class Diagram

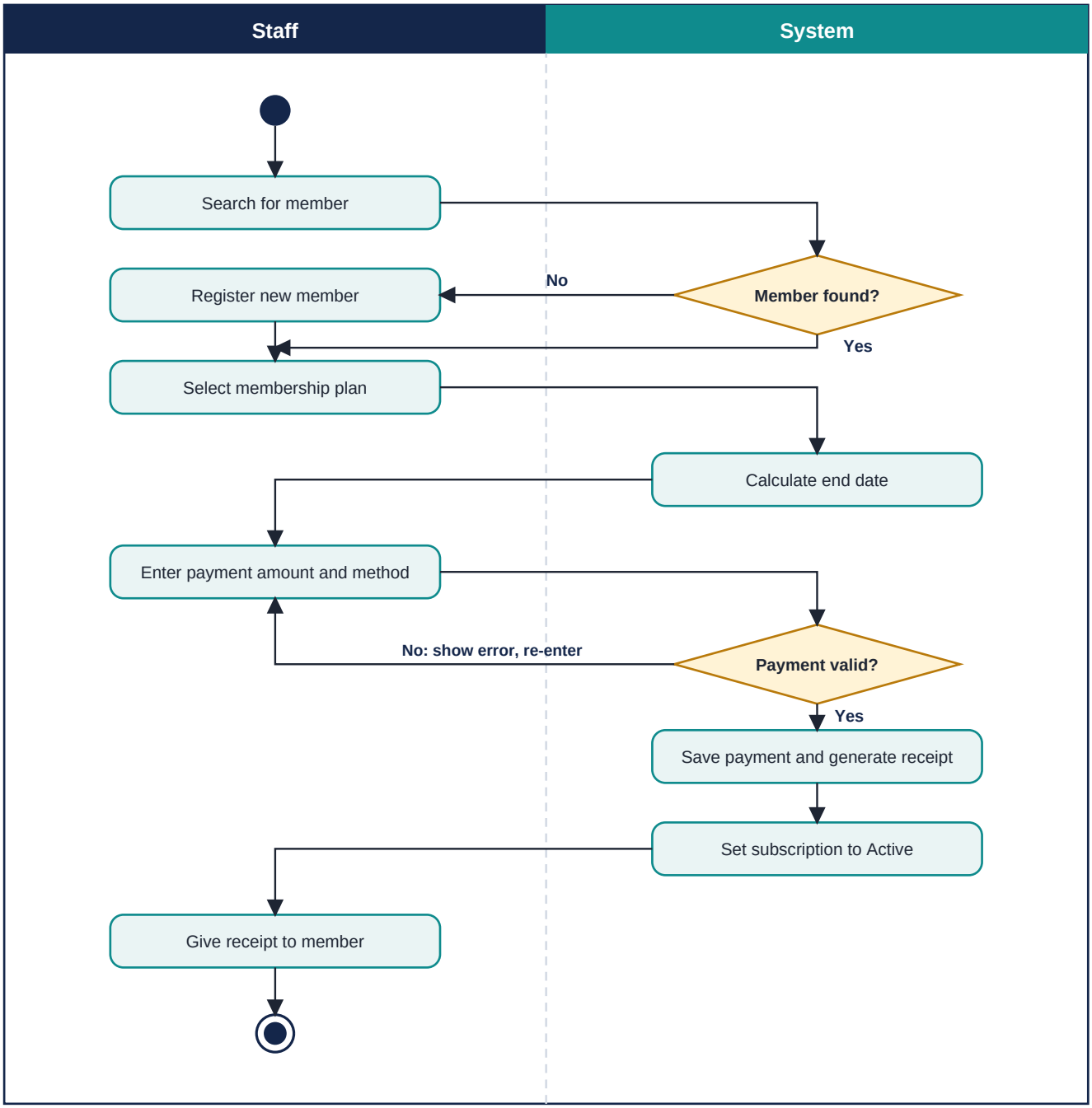
Purpose. The class diagram shows the main building blocks of the program: the classes, the data (attributes) and operations (methods) each one has, and how the classes are related. It is the bridge between the requirements and the code.



- **Relationships:** one member has many subscriptions over time (FR-11, FR-12); each subscription follows one membership plan and can have many payments (FR-15); a member has many check-ins (FR-19); a user records payments and check-ins.
- **ReportService** only reads data (dashed arrow) to produce the reports in FR-22 to FR-26. **Notation:** "-" private attribute, "+" public method, "1" exactly one, "0..*" zero or many.

3 Activity Diagram

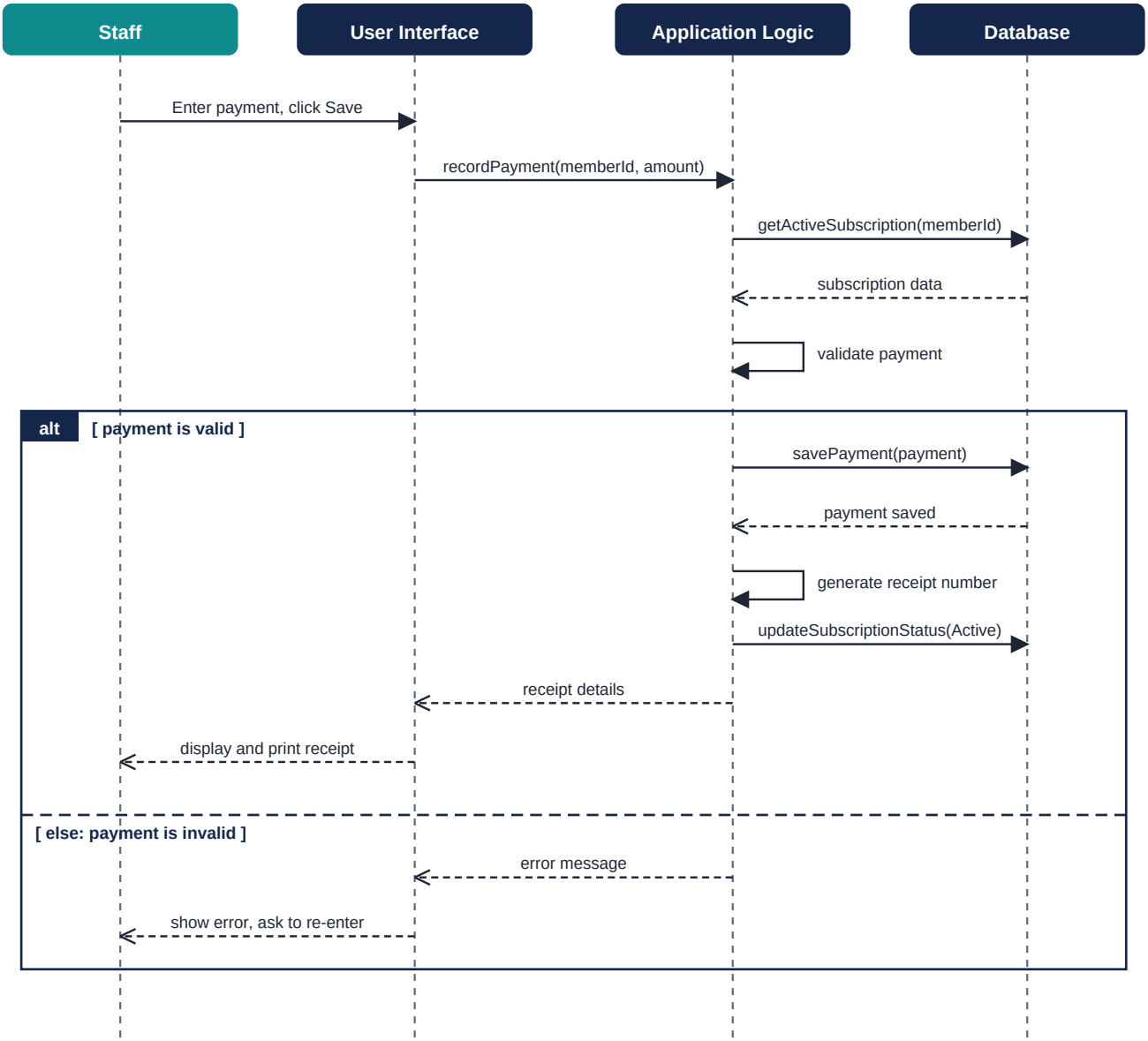
Purpose. The activity diagram shows the step-by-step flow of one important workflow: **renewing a subscription and recording a payment**, the most common daily task at the gym. Two swimlanes show what the staff member does and what the system does, and the diamonds show the decisions.



- **Decisions:** if the member is not found, staff register the member first (FR-05); if the payment is invalid, the system shows an error and staff re-enter it (NFR-09).
- **Automatic steps:** the system calculates the end date (FR-11), generates the receipt (FR-16), and sets the subscription to Active (FR-13), so staff do not need to do these by hand.

4 Sequence Diagram

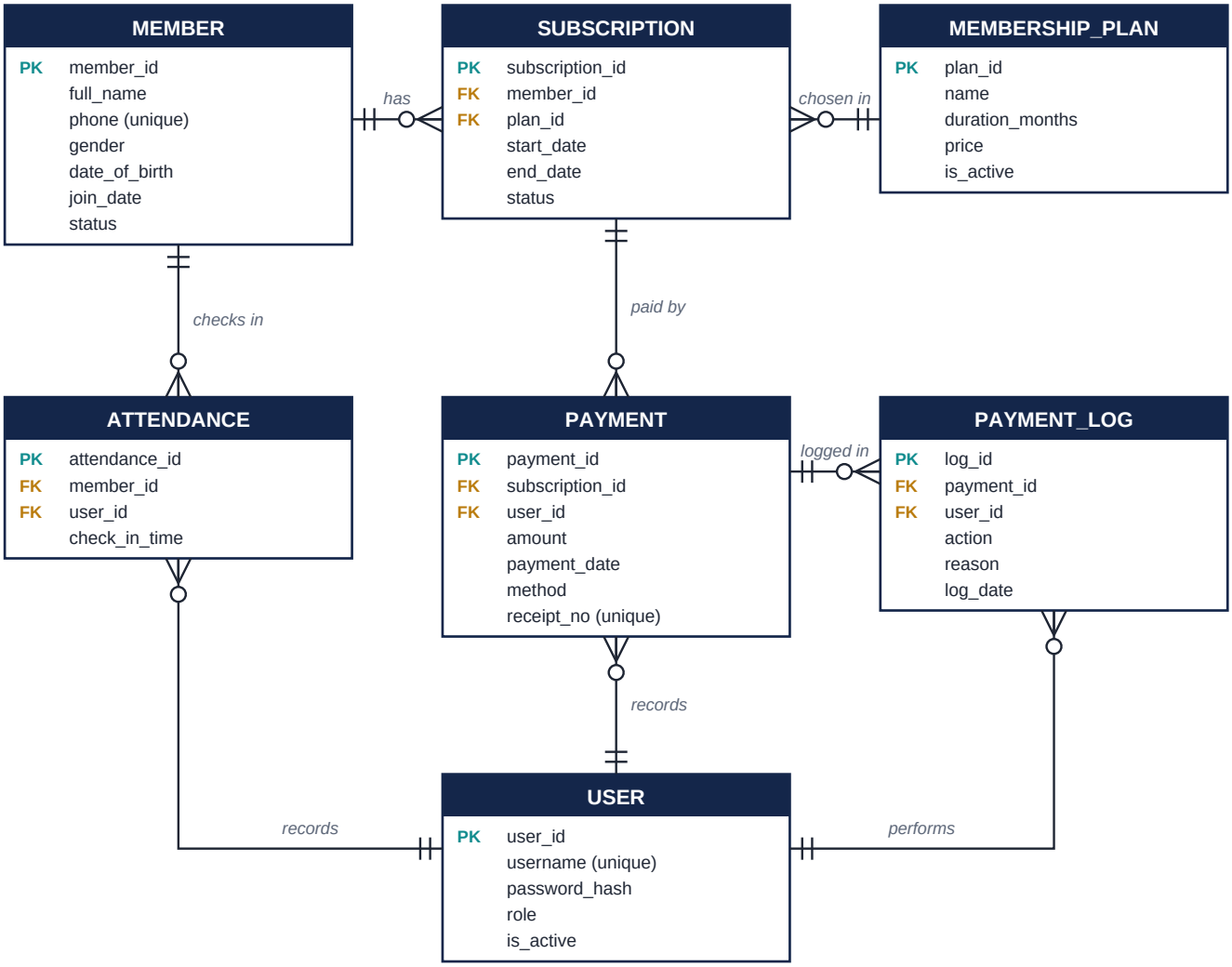
Purpose. The sequence diagram shows how the parts of the system exchange messages over time for one scenario: **recording a payment**. Time flows from top to bottom. It makes clear which part is responsible for each task, so the code is organized correctly.



- **Layers:** the staff member only talks to the user interface; the application logic checks and processes the data; only the database stores it. This separation supports NFR-10 (maintainability) and leads to the architecture in Part 4.
- **Alternatives:** the "alt" box shows both outcomes. If the payment is valid, it is saved, a receipt number is generated, and the subscription is updated (FR-15, FR-16). If not, an error message is returned and nothing is saved (NFR-09).
- **Notation:** solid arrows are requests, dashed arrows are replies, and an arrow that loops back means the part works on its own data.

5 Database (ER) Diagram

Purpose. The ER (entity-relationship) diagram shows the tables the database will have, their keys, and how they are linked. While the class diagram focuses on behavior, the ER diagram focuses on the data that must be stored permanently.



- **Keys:** PK is the primary key of a table; FK is a foreign key that links to another table. Lines show one-to-many relationships: a bar means "one" and a crow's foot with a circle means "zero or many".
- **Design decisions:** members have a *status* field so they are deactivated instead of deleted (FR-08); *phone* and *receipt_no* are unique (FR-09, FR-16); passwords are stored as hashes (NFR-07); and PAYMENT_LOG records every payment correction or cancellation by the Administrator (FR-18).
- **Consistency:** the tables match the classes in the class diagram, and every attribute used in the activity and sequence diagrams (amount, method, start and end dates, status) is stored here.